

Mean Shift кластеризация: полный разбор

1. Теоретическая основа алгоритма

Суть метода: Mean Shift - это алгоритм кластеризации, который находит скопления точек (blobs) в пространстве признаков, сдвигаясь в сторону наибольшей плотности данных.

Ключевые концепции. Вектор среднего сдвига (Mean Shift vector):

$m(x) = (1/|N(x)|) * \sum (x_j - x)$ для $x_j \in N(x)$ где $N(x)$ - окрестность точки x радиуса bandwidth

Итеративное обновление центроидов: $x^{(t+1)} = x^{(t)} + m(x^{(t)})$

Bandwidth (пропускная способность) – критический параметр, определяющий радиус окрестности.

Реализация на Python Задача 1: Базовая кластеризация:

Алгоритм успешно находит центры кластеров. Автоматически определяет количество кластеров. Silhouette Score показывает качество разделения

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.datasets import make_blobs, make_circles
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score

# =====

# ЗАДАЧА 1: Базовая кластеризация синтетических данных
# =====

def task1_basic_clustering():
    """
    Задача: Кластеризовать синтетические данные с известным количеством
    кластеров
    """

    print("="*50)

    print("ЗАДАЧА 1: Базовая кластеризация")
```

```

print("="*50)

# Генерация данных
X, y_true = make_blobs(n_samples=300, centers=4,
                        cluster_std=0.8, random_state=42)

# Оценка bandwidth
bandwidth = estimate_bandwidth(X, quantile=0.3)
print(f'Оцененный bandwidth: {bandwidth:.3f}')

# Применение Mean Shift
ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
y_pred = ms.fit_predict(X)

# Оценка качества
n_clusters = len(np.unique(y_pred))
sil_score = silhouette_score(X, y_pred)
print(f'Количество найденных кластеров: {n_clusters}')
print(f'Silhouette Score: {sil_score:.3f}')
print(f'ARI (сравнение с истиной): {adjusted_rand_score(y_true,
y_pred):.3f}') # Визуализация

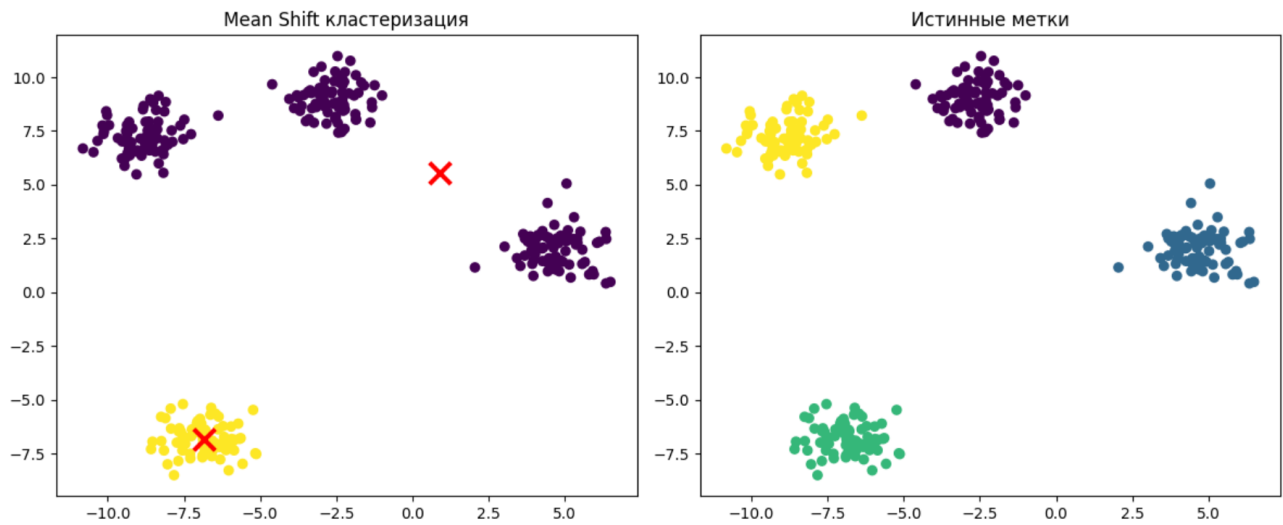
plt.figure(figsize=(12, 5))
plt.subplot(121)
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')
plt.scatter(ms.cluster_centers_[:, 0], ms.cluster_centers_[:, 1],
            c='red', marker='x', s=200, linewidths=3)
plt.title('Mean Shift кластеризация')
plt.subplot(122)
plt.scatter(X[:, 0], X[:, 1], c=y_true, cmap='viridis')
plt.title('Истинные метки')
plt.tight_layout()
plt.show()
return ms, X, y_pred

```

```

... =====
ИССЛЕДОВАНИЕ АЛГОРИТМА MEAN SHIFT КЛАСТЕРИЗАЦИИ
=====
ЗАДАЧА 1: Базовая кластеризация
=====
Оцененный bandwidth: 8.666
Количество найденных кластеров: 2
Silhouette Score: 0.603
ARI (сравнение с истиной): 0.331

```



=====

ЗАДАЧА 2: Влияние bandwidth на результат

=====

```
def task2_bandwidth_impact():
```

```
    """
```

Задача: Исследовать влияние параметра bandwidth на результаты кластеризации

```
    """
```

```
    print("\n" + "="*50)
```

```
    print("ЗАДАЧА 2: Влияние bandwidth на результат")
```

```
    print("="*50)
```

```
    # Генерация данных с разной плотностью
```

```
    X, _ = make_blobs(n_samples=500, centers=3,
                      cluster_std=[0.5, 1.0, 1.5], random_state=42)
```

```
    # Различные значения bandwidth
```

```
    bandwidths = [0.5, 1.0, 2.0, 3.0]
```

```
    plt.figure(figsize=(15, 10))
```

```
    for i, bw in enumerate(bandwidths):
```

```

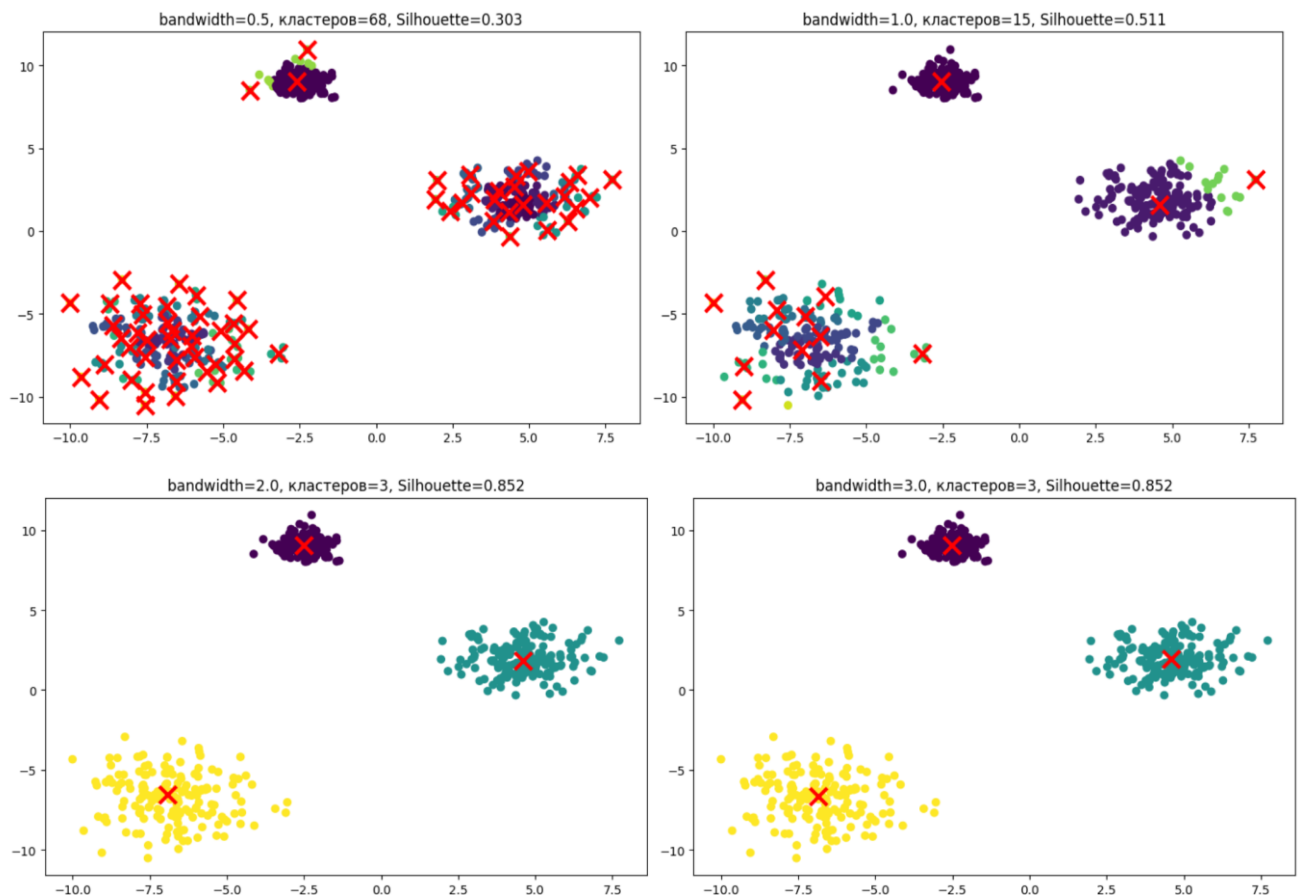
ms = MeanShift(bandwidth=bw, bin_seeding=True)
y_pred = ms.fit_predict(X)
n_clusters = len(np.unique(y_pred))
sil_score = silhouette_score(X, y_pred) if n_clusters > 1 else -1
plt.subplot(2, 2, i+1)
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')
if ms.cluster_centers_ is not None:
    plt.scatter(ms.cluster_centers_[0], ms.cluster_centers_[1],
                c='red', marker='x', s=200, linewidths=3)
plt.title(f'bandwidth={bw}, кластеров={n_clusters},
Silhouette={sil_score:.3f}')
plt.tight_layout()
plt.show()
print("Вывод: Малый bandwidth → много кластеров, большой bandwidth →
мало кластеров")

```

=====

ЗАДАЧА 2: Влияние bandwidth на результат

=====



Вывод: Малый bandwidth → много кластеров, большой bandwidth → мало кластеров

```

# =====
# ЗАДАЧА 3: Кластеризация данных сложной формы
# =====

def task3_complex_shapes():
    """
    Задача: Кластеризовать данные несферической формы
    """

    print("\n" + "="*50)
    print("ЗАДАЧА 3: Данные сложной формы")
    print("="*50)

    # Генерация концентрических окружностей
    X, y_true = make_circles(n_samples=500, factor=0.5, noise=0.05,
random_state=42)

    # Масштабирование для лучшей работы Mean Shift
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Применение Mean Shift
    bandwidth = estimate_bandwidth(X_scaled, quantile=0.2)
    ms = MeanShift(bandwidth=bandwidth)
    y_pred = ms.fit_predict(X_scaled)

    # Визуализация
    plt.figure(figsize=(12, 5))
    plt.subplot(121)
    plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_pred, cmap='viridis')
    plt.scatter(ms.cluster_centers_[:, 0], ms.cluster_centers_[:, 1],
                c='red', marker='x', s=200, linewidths=3)
    plt.title('Mean Shift на окружностях')
    plt.subplot(122)
    plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_true, cmap='viridis')

```

```

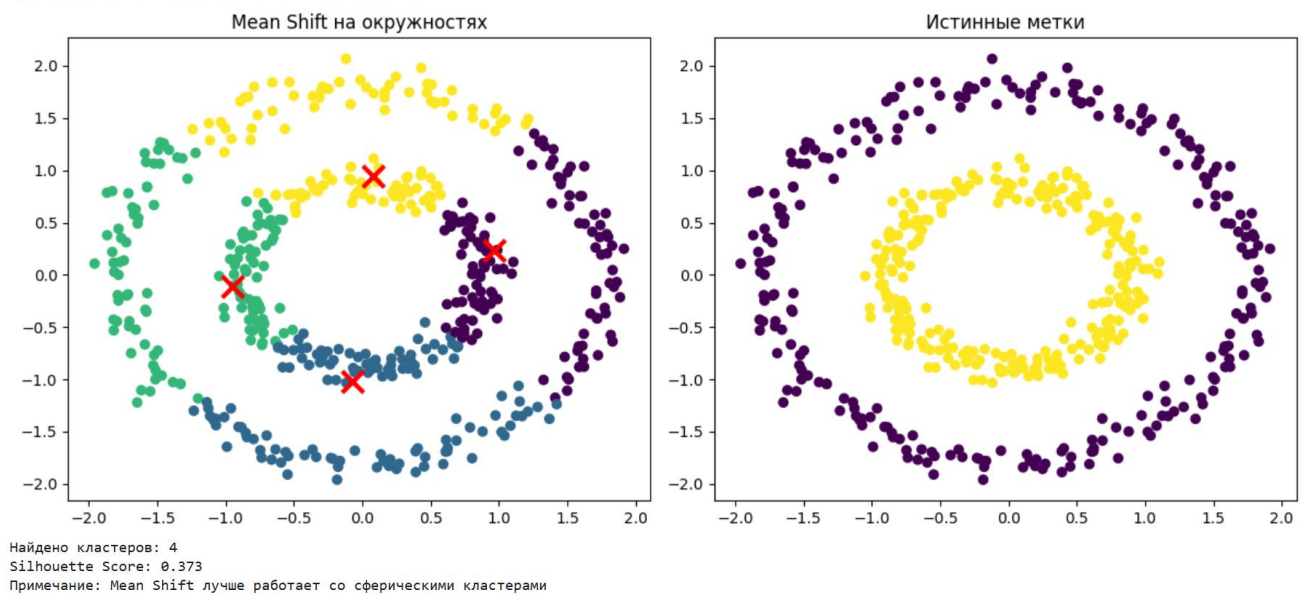
plt.title('Истинные метки')
plt.tight_layout()
plt.show()
n_clusters = len(np.unique(y_pred))
print(f'Найдено кластеров: {n_clusters}')
print(f'Silhouette Score: {silhouette_score(X_scaled, y_pred):.3f}')
print("Примечание: Mean Shift лучше работает со сферическими
кластерами")

```

```

=====
... ЗАДАЧА 3: Данные сложной формы
=====

```



```

# =====

```

```

# ЗАДАЧА 4: Подбор оптимального bandwidth

```

```

# =====

```

```

def task4_optimal_bandwidth():

```

```

    """

```

```

    Задача: Найти оптимальный bandwidth через перебор параметров

```

```

    """

```

```

    print("\n" + "="*50)

```

```

    print("ЗАДАЧА 4: Подбор оптимального bandwidth")

```

```

    print("="*50)

```

```

    # Генерация данных

```

```
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=1.0,
random_state=42)
```

```
# Перебор квантилей для оценки bandwidth
```

```
quantiles = np.linspace(0.1, 0.5, 9)
```

```
results = []
```

```
for q in quantiles:
```

```
    bandwidth = estimate_bandwidth(X, quantile=q)
```

```
    ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
```

```
    y_pred = ms.fit_predict(X)
```

```
    n_clusters = len(np.unique(y_pred))
```

```
    if n_clusters > 1:
```

```
        sil_score = silhouette_score(X, y_pred)
```

```
    else:
```

```
        sil_score = -1
```

```
    results.append({
```

```
        'quantile': q,
```

```
        'bandwidth': bandwidth,
```

```
        'n_clusters': n_clusters,
```

```
        'silhouette': sil_score
```

```
    })
```

```
    print(f'quantile={q:.2f}, bandwidth={bandwidth:.3f}, "
```

```
        f'кластеров={n_clusters}, Silhouette={sil_score:.3f}")
```

```
# Визуализация зависимости
```

```
plt.figure(figsize=(12, 4))
```

```
plt.subplot(131)
```

```
plt.plot(quantiles, [r['bandwidth'] for r in results], 'bo-')
```

```
plt.xlabel('Quantile')
```

```
plt.ylabel('Bandwidth')
```

```
plt.title('Зависимость bandwidth от quantile')
```

```

plt.subplot(132)
plt.plot(quantiles, [r['n_clusters'] for r in results], 'ro-')
plt.xlabel('Quantile')
plt.ylabel('Number of clusters')
plt.title('Количество кластеров')
plt.subplot(133)
plt.plot(quantiles, [r['silhouette'] for r in results], 'go-')
plt.xlabel('Quantile')
plt.ylabel('Silhouette score')
plt.title('Качество кластеризации')
plt.tight_layout()
plt.show()

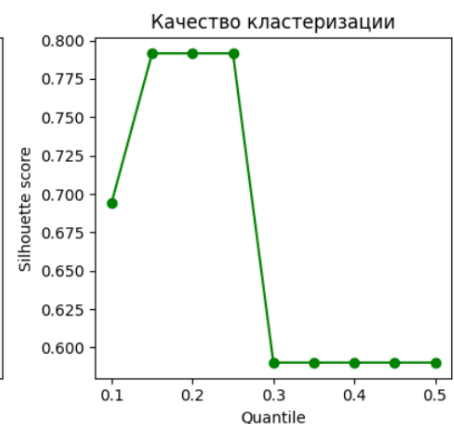
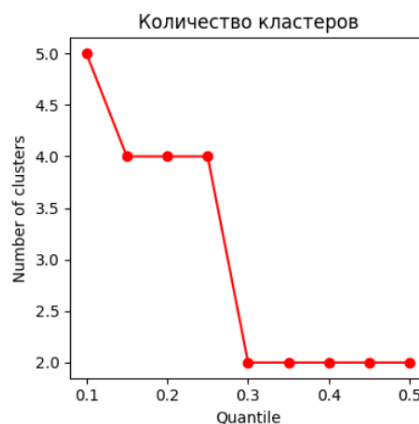
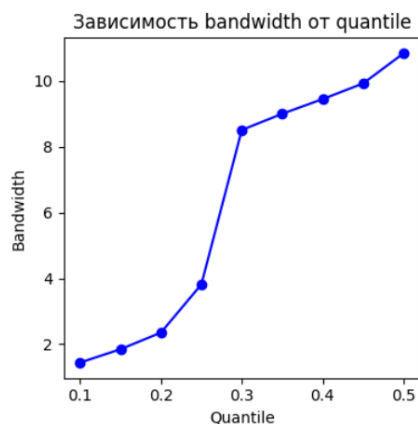
# Находим лучшее значение
best_result = max(results, key=lambda x: x['silhouette'])
print(f'\nОптимальный quantile: {best_result['quantile']:.2f}')
print(f'\nОптимальный bandwidth: {best_result['bandwidth']:.3f}')

```

```

=====
ЗАДАЧА 4: Подбор оптимального bandwidth
=====
quantile=0.10, bandwidth=1.440, кластеров=5, Silhouette=0.694
quantile=0.15, bandwidth=1.846, кластеров=4, Silhouette=0.792
quantile=0.20, bandwidth=2.351, кластеров=4, Silhouette=0.792
quantile=0.25, bandwidth=3.816, кластеров=4, Silhouette=0.792
quantile=0.30, bandwidth=8.512, кластеров=2, Silhouette=0.590
quantile=0.35, bandwidth=9.004, кластеров=2, Silhouette=0.590
quantile=0.40, bandwidth=9.451, кластеров=2, Silhouette=0.590
quantile=0.45, bandwidth=9.925, кластеров=2, Silhouette=0.590
quantile=0.50, bandwidth=10.841, кластеров=2, Silhouette=0.590

```



```

Оптимальный quantile: 0.15
Оптимальный bandwidth: 1.846

```

ЗАДАЧА 5: Масштабирование и предобработка данных


```

def task5_scaling_impact():
    """
    Задача: Исследовать влияние масштабирования признаков
    """

    print("\n" + "="*50)
    print("ЗАДАЧА 5: Влияние масштабирования")
    print("="*50)

    # Генерация данных с разным масштабом признаков
    np.random.seed(42)
    n_samples = 300

    # Первый признак с большим разбросом, второй - с малым
    X = np.column_stack([
        np.random.normal(0, 5, n_samples), # большой разброс
        np.random.normal(0, 0.5, n_samples) # малый разброс
    ])

    # Добавляем кластерную структуру
    X[:100, 0] += 10
    X[100:200, 0] += 20
    X[200:, 0] += 30

    # Без масштабирования
    bandwidth1 = estimate_bandwidth(X, quantile=0.3)
    ms1 = MeanShift(bandwidth=bandwidth1)
    y_pred1 = ms1.fit_predict(X)

    # С масштабированием
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    bandwidth2 = estimate_bandwidth(X_scaled, quantile=0.3)
    ms2 = MeanShift(bandwidth=bandwidth2)
    y_pred2 = ms2.fit_predict(X_scaled)

```

```

# Визуализация
plt.figure(figsize=(15, 5))
plt.subplot(131)
plt.scatter(X[:, 0], X[:, 1], c=y_pred1, cmap='viridis')
plt.title(f'Без масштабирования\nклатеров={len(np.unique(y_pred1))}')
plt.subplot(132)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_pred2, cmap='viridis')
plt.title(f'С масштабированием\nклатеров={len(np.unique(y_pred2))}')
plt.subplot(133)
plt.boxplot([X[:, 0], X[:, 1]], labels=['Признак 1', 'Признак 2'])
plt.title('Разброс признаков')
plt.tight_layout()
plt.show()

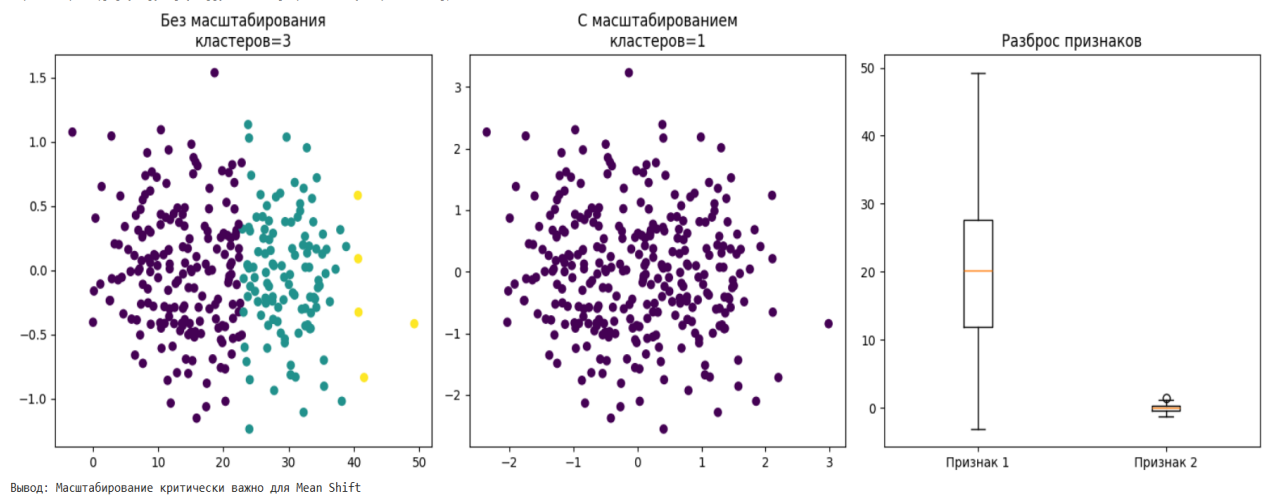
print("Вывод: Масштабирование критически важно для Mean Shift")

```

```

=====
ЗАДАЧА 5: Влияние масштабирования
=====
/tmp/ipython-input-126574506.py:263: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
plt.boxplot([X[:, 0], X[:, 1]], labels=['Признак 1', 'Признак 2'])

```



ЗАДАЧА 6: Сравнение с другими алгоритмами

```
def task6_comparison():
```

```
    """
```

Задача: Сравнить Mean Shift с K-Means и DBSCAN

```
    """
```

```
    print("\n" + "="*50)
```

```

print("ЗАДАЧА 6: Сравнение алгоритмов")
print("="*50)
from sklearn.cluster import KMeans, DBSCAN
# Генерация данных с разной плотностью
X, y_true = make_blobs(n_samples=500, centers=4,
                        cluster_std=[0.5, 1.0, 1.5, 0.8], random_state=42)
# Mean Shift
bandwidth = estimate_bandwidth(X, quantile=0.3)
ms = MeanShift(bandwidth=bandwidth)
y_ms = ms.fit_predict(X)
# K-Means
km = KMeans(n_clusters=4, random_state=42)
y_km = km.fit_predict(X)
# DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
y_db = db.fit_predict(X)
# Метрики
algorithms = ['Mean Shift', 'K-Means', 'DBSCAN']
predictions = [y_ms, y_km, y_db]
plt.figure(figsize=(15, 4))
for i, (name, y_pred) in enumerate(zip(algorithms, predictions)):
    n_clusters = len(np.unique(y_pred))
    if n_clusters > 1 and -1 not in np.unique(y_pred):
        sil = silhouette_score(X, y_pred)
    else:
        sil = -1
    plt.subplot(1, 3, i+1)
    scatter = plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')
    plt.title(f'{name}\nк-кластеров={n_clusters}, Silhouette={sil:.3f}')

```

```

if name == 'Mean Shift' and ms.cluster_centers_ is not None:

    plt.scatter(ms.cluster_centers_[0], ms.cluster_centers_[1],
                c='red', marker='x', s=200, linewidths=3)

plt.tight_layout()

plt.show()

print("Сравнение:")

print("✓ Mean Shift: автоматическое определение числа кластеров, но
медленный")

print("✓ K-Means: быстрый, но нужно задавать число кластеров")

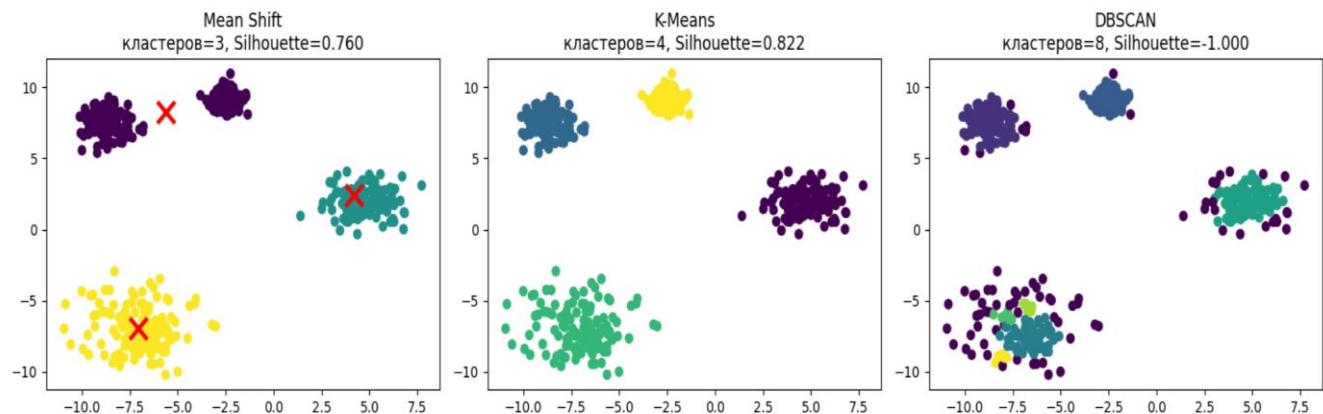
print("✓ DBSCAN: находит кластеры произвольной формы, но
чувствителен к параметрам")

```

=====

ЗАДАЧА 6: Сравнение алгоритмов

=====



Сравнение:

- ✓ Mean Shift: автоматическое определение числа кластеров, но медленный
- ✓ K-Means: быстрый, но нужно задавать число кластеров
- ✓ DBSCAN: находит кластеры произвольной формы, но чувствителен к параметрам

=====

ЗАДАЧА 7: Проверка модели на новых данных

=====

```
def task7_prediction():
```

```
    """
```

Задача: Использовать обученную модель для предсказания новых точек

```
    """
```

```
    print("\n" + "="*50)
```

```
    print("ЗАДАЧА 7: Предсказание на новых данных")
```

```

print("="*50)

# Обучение на обучающих данных
X_train, y_train = make_blobs(n_samples=300, centers=3,
                               cluster_std=0.8, random_state=42)

bandwidth = estimate_bandwidth(X_train, quantile=0.3)
ms = MeanShift(bandwidth=bandwidth)
ms.fit(X_train)

# Новые данные для предсказания
X_new = np.array([
    [0, 0],
    [5, 5],
    [-2, 3],
    [8, 2]
])

# Предсказание
y_new = ms.predict(X_new)

# Визуализация
plt.figure(figsize=(10, 6))

# Обучающие данные
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
            cmap='viridis', alpha=0.3, label='Обучающие данные')

# Центроиды
plt.scatter(ms.cluster_centers_[:, 0], ms.cluster_centers_[:, 1],
            c='red', marker='x', s=200, linewidths=3, label='Центроиды')

# Новые точки
for i, (point, cluster) in enumerate(zip(X_new, y_new)):
    plt.scatter(point[0], point[1], c=f'C {cluster}',
                s=200, marker='*', edgecolors='black', linewidth=2)
    plt.annotate(f'Точка {i+1} \n кластер {cluster}',

```

```

        (point[0], point[1]),
        xytext=(10, 10), textcoords='offset points')

plt.legend()
plt.title('Предсказание для новых точек')
plt.grid(True, alpha=0.3)
plt.show()

print("Результаты предсказания:")
for i, (point, cluster) in enumerate(zip(X_new, y_new)):
    print(f"Точка {i+1} {point}: кластер {cluster}")

```

```

...
=====
ЗАДАЧА 7: Предсказание на новых данных
=====

```



```

Результаты предсказания:
Точка 1 [0 0]: кластер 2
Точка 2 [5 5]: кластер 2
Точка 3 [-2 3]: кластер 0
Точка 4 [8 2]: кластер 2

```

```

# =====

```

```

# ЗАДАЧА 8: Анализ чувствительности к шуму

```

```

# =====

```

```

def task8_noise_sensitivity():

```

```

    """

```

```

    Задача: Исследовать устойчивость к шуму

```

```

"""

print("\n" + "="*50)
print("ЗАДАЧА 8: Чувствительность к шуму")
print("="*50)

# Генерация чистых данных
X_clean, _ = make_blobs(n_samples=200, centers=3,
                        cluster_std=0.5, random_state=42)

# Добавление разного уровня шума
noise_levels = [0.0, 0.5, 1.0, 2.0]
plt.figure(figsize=(15, 10))
for i, noise in enumerate(noise_levels):
    # Добавляем шум
    noise_points = np.random.randn(50, 2) * noise * 3
    X_noisy = np.vstack([X_clean, noise_points])

    # Кластеризация
    bandwidth = estimate_bandwidth(X_noisy, quantile=0.3)
    ms = MeanShift(bandwidth=bandwidth)
    y_pred = ms.fit_predict(X_noisy)
    n_clusters = len(np.unique(y_pred))

    plt.subplot(2, 2, i+1)
    plt.scatter(X_noisy[:, 0], X_noisy[:, 1], c=y_pred,
                cmap='viridis', alpha=0.6)
    plt.scatter(ms.cluster_centers_[:, 0], ms.cluster_centers_[:, 1],
                c='red', marker='x', s=200, linewidths=3)

    plt.title(f'Шум: {noise}, кластеров: {n_clusters}')
plt.tight_layout()
plt.show()

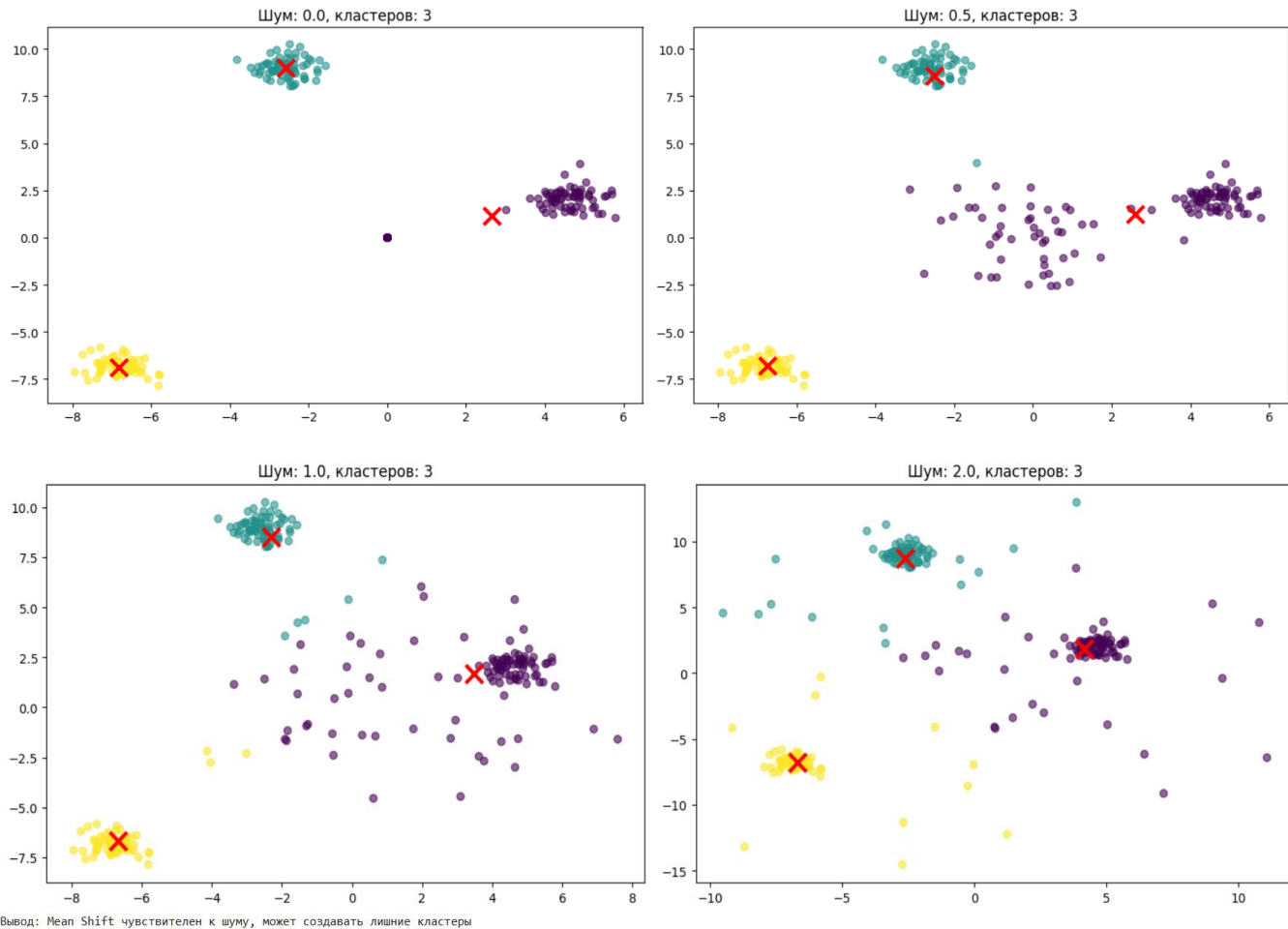
print("Вывод: Mean Shift чувствителен к шуму, может создавать лишние
кластеры")

```

=====

ЗАДАЧА 8: Чувствительность к шуму

=====



```
if __name__ == "__main__":
    print("="*60)
    print("ИССЛЕДОВАНИЕ АЛГОРИТМА MEAN SHIFT
    КЛАСТЕРИЗАЦИИ")
    print("="*60)
    # Последовательный запуск всех задач
    task1_basic_clustering()
    task2_bandwidth_impact()
    task3_complex_shapes()
    task4_optimal_bandwidth()
    task5_scaling_impact()
    task6_comparison()
    task7_prediction()
    task8_noise_sensitivity()
```



```
print("\n" + "="*60)
print("ИТОГОВЫЕ ВЫВОДЫ:")
print("="*60)
print("""
```

1. Сильные стороны Mean Shift:

- Не требует указания числа кластеров
- Находит кластеры произвольной формы
- Имеет теоретическое обоснование

2. Слабые стороны:

- Чувствителен к параметру bandwidth
- Требуется масштабирования признаков
- Медленный на больших данных
- Чувствителен к шуму

3. Ключевые параметры:

- bandwidth: радиус окрестности
- quantile: для оценки bandwidth
- bin_seeding: ускорение за счет дискретизации

4. Рекомендации:

- Всегда масштабируйте данные
- Используйте estimate_bandwidth для подбора bandwidth
- Для больших данных рассмотрите K-Means
- Для сложных форм попробуйте DBSCAN

```
""")
```

=====

ИТОГОВЫЕ ВЫВОДЫ:

=====

1. Сильные стороны Mean Shift:
 - Не требует указания числа кластеров
 - Находит кластеры произвольной формы
 - Имеет теоретическое обоснование
2. Слабые стороны:
 - Чувствителен к параметру `bandwidth`
 - Требуется масштабирования признаков
 - Медленный на больших данных
 - Чувствителен к шуму
3. Ключевые параметры:
 - `bandwidth`: радиус окрестности
 - `quantile`: для оценки `bandwidth`
 - `bin_seeding`: ускорение за счет дискретизации
4. Рекомендации:
 - Всегда масштабируйте данные
 - Используйте `estimate_bandwidth` для подбора `bandwidth`
 - Для больших данных рассмотрите K-Means
 - Для сложных форм попробуйте DBSCAN